

微网与外部电网电力调控策略

总体思路文档

2026 年高教社杯全国大学生数学建模竞赛 C 题

1 题目背景与总体方向

在“双碳”目标与新型电力系统建设的大背景下，以光伏为代表的分布式新能源大规模接入居民小区与产业园区。光伏发电受天气影响具有明显的随机性与波动性，其出力高峰往往与居民用电高峰错位，单纯依赖外部电网供电既不经济，也难以消纳富余的绿色电能。由光伏阵列、储能电池、小区负载与外部电网共同构成的微网（Microgrid），通过储能“低充高放”实现电能的时间平移，被视为提升新能源消纳率、降低用电成本、增强供电可靠性的关键技术手段。然而，储能电池容量与功率有限、充放电存在效率损耗、荷电状态必须维持在安全区间，如何在满足供电不中断这一硬约束的前提下制定经济最优的购电与充放电计划，是微网能量管理系统的核心问题。

本题围绕一个装机光伏、配置 12000 kWh 储能电池的居民小区微网展开，以 10 分钟为一个调度时段、全天 144 个时段离散建模，四个问题层层递进：问题一研究电价与负载逐日相同的典型日确定型调度，刻画储能套利的基本机理；问题二将其扩展到全年 365 天的真实负载与光伏，处理储电量跨日连续与供电可靠性约束；问题三引入光伏预报误差，采用“0 点计划、6/12/18 点滚动调整”的两阶段调度，并通过偏差考核与紧急购电机刻画不确定性成本；问题四将固定电价替换为实时波动电价，分别重算问题二、三，形成“电价是否波动 $\times$ 是否滚动”的四场景对比。四问在数据规模上由单日到全年、在信息结构上由完全确定到预报不确定、在价格机制上由固定到波动，构成一条由机理到应用、由理想模型到真实环境的完整研究主线。本题在方法上属于带约束的数学规划问题，核心工具是线性规划与后退时域滚动优化。

2 总体研究思路图

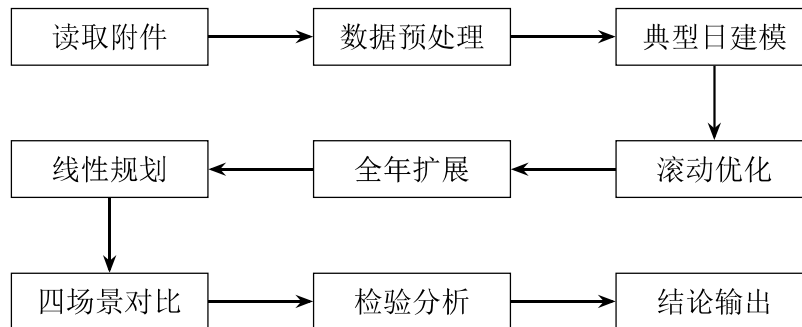


图 1: 总体研究思路图

3 各题求解思路

3.1 问题一：典型日确定型调度

问题背景与具体分析

问题一是全部研究的机理基础。给定典型日 144 时段的固定电价、小区负载与光伏预测功率，要求制定当日购电与储能充放电计划，使购电费最小，并满足储电量日首尾相等（日循环）、荷电状态处于安全区间等约束。其输入为单日 144×4 的规整时序数据，输出为逐时段购电量、充放电量与储电量。难点在于刻画充放电效率损耗、SOC 时序耦合与功率、容量边界，并证明充放电不会同时发生。

方法适用性分析

处理单日时序调度的候选方法包括贪心规则、动态规划与线性规划。贪心规则（如“电价低于阈值就充、高于阈值就放”）直观但阈值难以兼顾容量与功率约束，且无法保证全局最优；动态规划以储电量为状态可求最优，但连续状态需离散、精度与计算量相互制约；线性规划将购电、充放电、储电量全部设为连续决策变量，所有约束与目标均为线性，可由单纯形/内点法求得全局最优，且充放互斥可由往返效率小于 1 自动保证，无需引入整数变量，最契合本问。

思路图

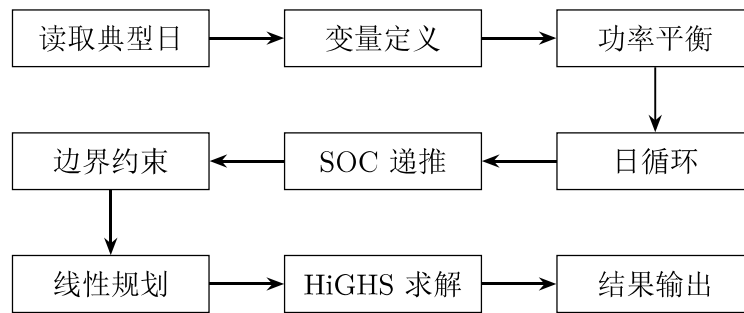


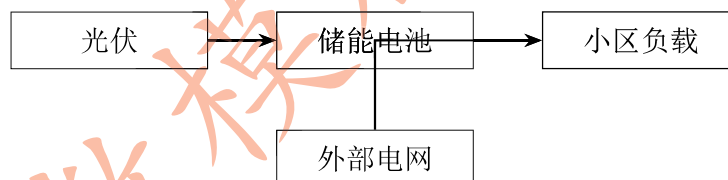
图 2: 问题一思路图

建模与模型示意

以 g_t, c_t, d_t 表示购电、充、放电功率， E_t 表示储电量，核心模型为

$$\min \sum_{t=1}^{144} p_t g_t \Delta t, \quad g_t + d_t - c_t = L_t - P_t, \quad E_t = E_{t-1} + \eta c_t \Delta t - \frac{d_t \Delta t}{\eta}, \quad (1)$$

并约束 $E_{144} = E_0$ (日循环)、 $1200 \leq E_t \leq 10800$ 、 $0 \leq c_t, d_t \leq 5000$ 。由于往返效率 $\eta^2 = 0.81 < 1$ ，同时充放必然增加损耗，最优解自动满足 $\min(c_t, d_t) = 0$ 。



核心代码与结果

```

1 # -*- coding: utf-8 -*-
2 """
3 问题一：典型日确定型经济调度
4 =====
5 每天电价与负载相同（附件1），光伏为一天预测；微网供电不低于负载，
6 储能0:00与24:00储电量相等（值自由）。建立单周期线性规划(LP)，
7 最小化全天购电费，输出计划购电/充放电策略、result1.xlsx与结果图。
8 """
9 import os, sys
10 import numpy as np
11 import pandas as pd
12 import matplotlib.pyplot as plt
13
14 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
15 import 微网公共模块 as M
16

```

```

17 BASE = os.path.dirname(os.path.abspath(__file__))
18
19 # 自然日重排：模板顺序 k=143(0:00-0:10) 移到最前，使图轴 0->24h
20 NAT = [143] + list(range(143))
21 HOURS = np.arange(144) / 6.0 # 自然顺序下每时段起点小时 0..23.833
22
23 # ===== 第一阶段：计算 =====
24 td = M.load_typical_day()
25 price, load, pv = td['price'], td['load'], td['pv_fcst']
26 net = load - pv # 净负荷 kW
27
28 # 日循环 LP (E0=E144, 值自由)
29 res = M.solve_dispatch(price, load, pv, E_start=None, cycle=True, emergency=False)
30 g, c, d, E = res['g'], res['c'], res['d'], res['E']
31 fee = res['cost']
32 buy_qty = g.sum() * M.DT
33 ch_qty = c.sum() * M.DT
34 dis_qty = d.sum() * M.DT
35
36 # 无储能对照：净负荷为正时直接购电
37 g_base = np.maximum(net, 0.0)
38 fee_base = float((price * g_base * M.DT).sum())
39 qty_base = g_base.sum() * M.DT
40 save_ratio = (fee_base - fee) / fee_base * 100
41
42 # 表1 指定时段购电量（模板顺序索引）
43 tab1_slots = ['10:00', '12:00', '14:00', '16:00', '18:00', '20:00']
44 tab1 = {s: g[M.slot_index_by_hm(s)] * M.DT for s in tab1_slots}
45 # 表2 六个4小时时段充放电量 + 首尾SOC
46 seg_names = ['0:00-4:00', '4:00-8:00', '8:00-12:00', '12:00-16:00', '16:00-20:00', '20:00-24:00']
47 tab2 = []
48 for i, nm in enumerate(seg_names):
49     seg = slice(i*24, (i+1)*24)
50     tab2.append((nm, c[seg].sum()*M.DT, d[seg].sum()*M.DT))
51
52 # 打印统计量（供论文引用）
53 print('='*60)
54 print('问题一求解结果')
55 print('='*60)
56 print(M.stats_line('电价', price)); print(M.stats_line('小区负载', load)); print(M.stats_line('光伏预测', pv))
57 print(M.stats_line('购电功率', g)); print(M.stats_line('SOC', E))
58 print(f'全天购电量: {buy_qty:.2f} kWh')
59 print(f'全天购电费: {fee:.2f} 元')
60 print(f'充电总量: {ch_qty:.2f} kWh, 放电总量: {dis_qty:.2f} kWh, 综合效率: {dis_qty/ch_qty:.4f}')
61 print(f'0:00与24:00储电量: {E[0]:.2f} kWh')
62 print(f'无储能购电费: {fee_base:.2f} 元, 储能节省: {fee_base-fee:.2f} 元 ({save_ratio:.2f}%)')
63 print('表1指定时段购电量(kWh):', {k: round(v,3) for k,v in tab1.items()})
64 print('表2各时段充电/放电(kWh):', [(n, round(a,2), round(b,2)) for n,a,b in tab2])
65 # 校验
66 assert abs(E[0]-E[-1]) < 1e-6, '首尾SOC不等'
67 assert np.max(np.minimum(c, d)) < 1e-6, '存在同时充放'
68 bal = pv + g + d - load - c
69 assert np.abs(bal).max() < 1e-6, '功率不平衡'

```

```

70 print('校验通过：功率平衡、首尾相等、无同时充放')
71
72 # 保存预处理数据 & 结果CSV
73 pre = pd.DataFrame({'时段标签': M.result_slot_labels(), '电价': price, '小区负载': load,
74                    '光伏预测功率': pv, '净负荷': net, '计划购电功率': g,
75                    '充电功率': c, '放电功率': d})
76 pre.to_csv(os.path.join(M.SOLVE_DIR, '预处理数据.csv'), index=False, encoding='utf-8-sig')
77 M.save_csv(pre, BASE, '问题一_典型日调度结果.csv')
78
79 # 写 result1.xlsx
80 out_path = M.write_result1(BASE, g, c, d, E)
81 print('已写出', out_path)
82
83 # ===== 第二阶段：绘图 =====
84 # 图1：典型日电价、负载、光伏三联曲线
85 fig, ax1 = plt.subplots(figsize=(10, 4.5))
86 ax1.plot(HOURS, price[NAT], color=M.C_PRICE, lw=1.8, label='电价')
87 ax1.set_xlabel('时刻/h'); ax1.set_ylabel('电价/(元/kWh)', color=M.C_PRICE)
88 ax1.tick_params(axis='y', labelcolor=M.C_PRICE); M.despine(ax1); M.grid(ax1)
89 ax2 = ax1.twinx()
90 ax2.plot(HOURS, load[NAT], color=M.C_LOAD, lw=1.8, label='小区负载')
91 ax2.plot(HOURS, pv[NAT], color=M.C_PV, lw=1.8, label='光伏发电')
92 ax2.set_ylabel('功率/kW'); M.despine(ax2)
93 lines = ax1.get_lines() + ax2.get_lines()
94 ax1.legend(lines, [l.get_label() for l in lines], loc='upper_left', ncol=3, frameon=False)
95 ax1.set_xlim(0, 24); ax1.set_xticks(range(0, 25, 2))
96 M.save_fig(fig, BASE, '图1_典型日电价负载光伏曲线.png')
97
98 # 图2：计划购电量与净负荷、电价关系
99 fig, ax = plt.subplots(figsize=(10, 4.5))
100 ax.fill_between(HOURS, 0, net[NAT], color=M.C_LOAD, alpha=0.18, label='净负荷(负载-光伏)')
101 ax.plot(HOURS, g[NAT], color=M.C_GRID, lw=1.8, label='计划购电功率')
102 ax.set_xlabel('时刻/h'); ax.set_ylabel('功率/kW'); M.despine(ax); M.grid(ax)
103 ax.set_xlim(0, 24); ax.set_xticks(range(0, 25, 2)); ax.legend(frameon=False, loc='upper_left')
104 axb = ax.twinx(); axb.plot(HOURS, price[NAT], color=M.C_PRICE, lw=1.3, ls='--', label='电价')
105 axb.set_ylabel('电价/(元/kWh)', color=M.C_PRICE); axb.tick_params(axis='y', labelcolor=M.
    C_PRICE); M.despine(axb)
106 M.save_fig(fig, BASE, '图2_计划购电量与净负荷电价.png')
107
108 # 图3：储能SOC轨迹
109 fig, ax = plt.subplots(figsize=(10, 4.2))
110 ax.plot(HOURS, E[NAT], color=M.C_SOC, lw=2.0, marker='o', ms=2.5, label='储电量SOC')
111 ax.axhline(M.SOC_MIN, color='gray', ls=':', lw=1, label='SOC下限1200')
112 ax.axhline(M.SOC_MAX, color='gray', ls='--', lw=1, label='SOC上限10800')
113 ax.fill_between(HOURS, M.SOC_MIN, M.SOC_MAX, color='gray', alpha=0.06)
114 ax.set_xlabel('时刻/h'); ax.set_ylabel('储电量/kWh'); M.despine(ax); M.grid(ax)
115 ax.set_xlim(0, 24); ax.set_xticks(range(0, 25, 2)); ax.legend(frameon=False, loc='upper_left')
116 M.save_fig(fig, BASE, '图3_储能SOC日内轨迹.png')
117
118 # 图4：充放电功率
119 fig, ax = plt.subplots(figsize=(10, 4.2))
120 ax.bar(HOURS, c[NAT]/6, width=0.12, color=M.C_CH, alpha=0.8, label='充电电量')
121 ax.bar(HOURS, -d[NAT]/6, width=0.12, color=M.C_DIS, alpha=0.8, label='放电电量')
122 ax.axhline(0, color='black', lw=0.8)
123 ax.set_xlabel('时刻/h'); ax.set_ylabel('每时段充(+)/放(-)电量/kWh'); M.despine(ax); M.grid(ax)
124 ax.set_xlim(0, 24); ax.set_xticks(range(0, 25, 2)); ax.legend(frameon=False, loc='upper_left')

```

```

125 M.save_fig(fig, BASE, '图4_储能充放电功率.png')
126
127 # 图5: 低充高放套利示意 (电价底色+充放)
128 fig, ax = plt.subplots(figsize=(10, 4.2))
129 colors = [M.C_CH if ci > 1e-6 else (M.C_DIS if di > 1e-6 else M.C_GRID)
130           for ci, di in zip(c[NAT], d[NAT])]
131 ax.scatter(HOURS, price[NAT], c=[M.C_CH if x>1e-6 else (M.C_DIS if y>1e-6 else '#bbbbbb')
132                               for x,y in zip(c[NAT],d[NAT])], s=18, alpha=0.8)
133 ax.plot(HOURS, price[NAT], color='#999999', lw=1.2)
134 ax.set_xlabel('时刻/h'); ax.set_ylabel('电价/(元/kWh)'); M.despine(ax); M.grid(ax)
135 ax.set_xlim(0,24); ax.set_xticks(range(0,25,2))
136 from matplotlib.patches import Patch
137 ax.legend(handles=[Patch(color=M.C_CH,label='主要充电时段'),Patch(color=M.C_DIS,label='主要放
    电时段'),
138                  Patch(color='#bbbbbb',label='待机时段')], frameon=False, loc='upper_left')
139 M.save_fig(fig, BASE, '图5_低充高放套利时段分布.png')
140
141 # 图6: 供电构成堆叠图 (光伏/购电/放电 供给)
142 fig, ax = plt.subplots(figsize=(10, 4.2))
143 ax.stackplot(HOURS, pv[NAT], g[NAT], d[NAT],
144             labels=['光伏发电','外网购电','储能放电'],
145             colors=[M.C_PV, M.C_GRID, M.C_DIS], alpha=0.85)
146 ax.plot(HOURS, load[NAT]+c[NAT], color='black', lw=1.5, ls='--', label='总需求(负载+充电)')
147 ax.set_xlabel('时刻/h'); ax.set_ylabel('功率/kW'); M.despine(ax); M.grid(ax)
148 ax.set_xlim(0,24); ax.set_xticks(range(0,25,2)); ax.legend(frameon=False, ncol=4, loc='upper_
    left')
149 M.save_fig(fig, BASE, '图6_微网供电构成堆叠图.png')
150
151 # 图7: 有无储能费用对比
152 fig, ax = plt.subplots(figsize=(6,4.5))
153 bars = ax.bar(['无储能','含储能优化'], [fee_base, fee], color=[M.C_LOAD, M.C_CH], width=0.5,
154             edgecolor='white')
155 for b,v in zip(bars,[fee_base,fee]):
156     ax.text(b.get_x()+b.get_width()/2, v+300, f'{v:.0f}元', ha='center', fontsize=10)
157 ax.set_ylabel('全天购电费/元'); M.despine(ax); M.grid(ax); ax.set_ylim(0, fee_base*1.12)
158 M.save_fig(fig, BASE, '图7_有无储能购电费对比.png')
159 print('问题一全部图表已生成并同步论文目录')

```

典型日购电量 59482.70 kWh、购电费 35118.60 元，相比无储能的 48052.05 元节省 26.92%，日首尾储电量均为 8550 kWh，充放电量之比恰为 0.81。

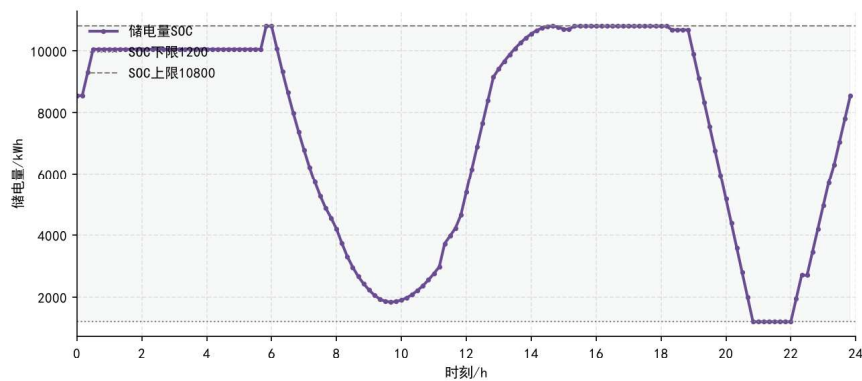


图 3: 问题一典型日储电量轨迹

3.2 问题二：全年确定型调度

问题背景与具体分析

问题二将单日模型扩展到全年 365 天、共 52560 个时段，电价逐日重复，负载与光伏取附件 2 实际数据，储电量跨日连续传递、年初为 6000 kWh，并引入 5 倍电价的紧急购电以保证供电不中断。难点在于大规模优化的高效求解、跨日 SOC 耦合，以及光伏大发超出消纳能力时的可行性。

方法适用性分析

逐日独立优化因无法预知次日而短视，动态规划在全年连续状态下代价高昂，随机规划在数据确定已知时属冗余；将全年展平为一条长序列做整体线性规划，SOC 递推天然实现跨日连续，借助稀疏矩阵与 HiGHS 可在数秒内求得全年全局最优，并通过紧急购电、弃光两个松弛变量保证恒可行，故为最优选择。

思路图

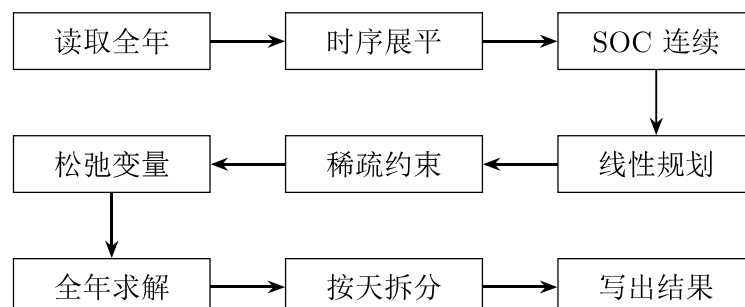


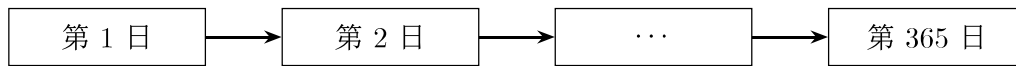
图 4: 问题二思路图

建模与模型示意

在问题一基础上增加紧急购电 e_t 与弃光 w_t ：

$$g_t + e_t + d_t - c_t - w_t = L_t - P_t, \quad 0 \leq w_t \leq P_t, \quad \min \sum_t (p_t g_t + 5p_t e_t) \Delta t, \quad (2)$$

SOC 递推跨全部日期连续，仅约束 $E_0 = 6000$ ，终端自由。



核心代码与结果

```

1 # -*- coding: utf-8 -*-
2 """
3 问题二：全年确定型调度（固定电价+紧急购电松弛）
4 =====
5 电价每天重复附件1曲线，负载/光伏为附件2全年实际；SOC跨日连续，
6 2025-01-01 0:00 初始 6000 kWh；供电不足时以 5 倍电价紧急购电（松弛），
7 光伏超出消纳能力时弃光。全年一次性线性规划，输出 2.1-12.31 的 result2.xlsx。
8 """
9 import os, sys, time
10 import numpy as np
11 import pandas as pd
12 import matplotlib.pyplot as plt
13
14 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
15 import 微网公共模块 as M
16
17 BASE = os.path.dirname(os.path.abspath(__file__))
18 NAT = [143] + list(range(143))
19 HOURS = np.arange(144) / 6.0
20
21 # ===== 第一阶段：计算 =====
22 td = M.load_typical_day(); yr = M.load_year_actual()
23 dates = yr['dates']; load_y = yr['load']; pv_y = yr['pv']
24 ND = len(dates)
25 price_day = td['price']
26 price_flat = np.tile(price_day, ND)
27 load_flat = load_y.reshape(-1); pv_flat = pv_y.reshape(-1)
28
29 t0 = time.time()
30 res = M.solve_dispatch(price_flat, load_flat, pv_flat, E_start=M.SOC_INIT,
31                        E_end=None, emergency=True)
32 print('全年LP求解耗时 %.2fs' % (time.time()-t0))
33 g = res['g'].reshape(ND, 144); c = res['c'].reshape(ND, 144)
34 d = res['d'].reshape(ND, 144); e = res['e'].reshape(ND, 144)
35 cur = res['cur'].reshape(ND, 144)
36 E_flat = res['E'] # 全年连续，长度 ND*144+1
37 E = np.array([E_flat[di*144:di*144+145] for di in range(ND)]) # (ND,145)
38
39 # 逐日统计
40 day_qty = g.sum(axis=1)*M.DT
    
```

```

41 day_fee = (price_day[None,:]*g*M.DT).sum(axis=1)
42 day_emg = e.sum(axis=1)*M.DT
43 day_cur = cur.sum(axis=1)*M.DT
44 day_charge = c.sum(axis=1)*M.DT; day_disch = d.sum(axis=1)*M.DT
45 # 无储能对照
46 base_g = np.maximum(load_y-pv_y, 0)
47 day_fee_base = (price_day[None,:]*base_g*M.DT).sum(axis=1)
48
49 print('='*60); print('问题二全年结果'); print('='*60)
50 print(M.stats_line('日购电量', day_qty)); print(M.stats_line('日购电费', day_fee))
51 print('全年购电量: %.2f 万kWh' % (day_qty.sum()/1e4))
52 print('全年购电费: %.2f 万元' % (day_fee.sum()/1e4))
53 print('全年紧急购电量: %.4f kWh, 紧急费: %.2f 元' % (day_emg.sum(), res['emg_fee']))
54 print('全年弃光电量: %.2f kWh' % day_cur.sum())
55 print('全年充电: %.2f 万kWh, 放电: %.2f 万kWh' % (day_charge.sum()/1e4, day_disch.sum()/1e4))
56 print('无储能全年费: %.2f 万元, 节省: %.2f%%' % (day_fee_base.sum()/1e4,
57       (day_fee_base.sum()-day_fee.sum())/day_fee_base.sum()*100))
58 print('SOC 全年范围: %.1f~%.1f' % (E.min(), E.max()))
59
60 # 输出区间 2.1-12.31
61 out_mask = np.array([(d.month >= 2) for d in dates])
62 dates_out = [dates[i] for i in range(ND) if out_mask[i]]
63 idx_out = np.where(out_mask)[0]
64 g_o, c_o, d_o, e_o, E_o = g[idx_out], c[idx_out], d[idx_out], e[idx_out], E[idx_out]
65 price_o = np.tile(price_day, (len(idx_out),1))
66 print('输出天数:', len(dates_out), dates_out[0], '-', dates_out[-1])
67
68 # 指定日期表1/2/3
69 key_dates = ['2025-03-20', '2025-06-21', '2025-09-23', '2025-12-21']
70 tab1_slots = ['10:00', '12:00', '14:00', '16:00', '18:00', '20:00']
71 key_info = {}
72 for kd in key_dates:
73     di = dates.index(pd.Timestamp(kd).date())
74     seg_ch, seg_dis = [], []
75     for s in range(6):
76         seg = slice(s*24, (s+1)*24)
77         seg_ch.append(c[di,seg].sum()*M.DT); seg_dis.append(d[di,seg].sum()*M.DT)
78     key_info[kd] = {
79         'idx': di,
80         'tab1': {s: g[di, M.slot_index_by_hm(s)]*M.DT for s in tab1_slots},
81         'qty': day_qty[di], 'fee': day_fee[di], 'emg': day_emg[di],
82         'ch': seg_ch, 'dis': seg_dis, 'E0': E[di,0], 'E24': E[di,-1]
83     }
84     print(f'\n指定日 {kd}: 日购电量{day_qty[di]:.1f} kWh, 日购电费{day_fee[di]:.1f} 元, 紧急{day_emg[di]:.2f}')
85     print('表1:', {s: round(v,2) for s,v in key_info[kd]['tab1'].items()})
86     print('充电[6段]:', [round(x,1) for x in seg_ch])
87     print('放电[6段]:', [round(x,1) for x in seg_dis], 'E0=%.1f E24=%.1f' % (E[di,0], E[di,-1]))
88
89 # 保存结果CSV
90 daily = pd.DataFrame({'日期': dates, '日购电量_kWh': day_qty, '日购电费_元': day_fee,
91       '日充电量_kWh': day_charge, '日放电量_kWh': day_disch,
92       '日紧急购电量_kWh': day_emg, '日弃光电量_kWh': day_cur,
93       '无储能购电费_元': day_fee_base})
94 M.save_csv(daily, BASE, '问题二_全年逐日调度统计.csv')

```

```

95
96 # 写 result2.xlsx
97 out_path = M.write_result_year(BASE, 'result2.xlsx', dates_out, g_o, c_o, d_o, E_o,
98                                     e_year=e_o, price_for_fee=price_o)
99 print('\n已写出', out_path)
100
101 # ===== 第二阶段：绘图 =====
102 day_idx = np.arange(ND)
103 # 图1：全年日购电费与无储能对照
104 fig, ax = plt.subplots(figsize=(11,4.2))
105 ax.plot(day_idx, day_fee_base/1e4, color='#bbbbbb', lw=1.2, label='无储能购电费')
106 ax.plot(day_idx, day_fee/1e4, color=M.C_LOAD, lw=1.4, label='储能优化购电费')
107 ax.fill_between(day_idx, day_fee/1e4, day_fee_base/1e4, color=M.C_PV, alpha=0.3, label='节省部分')
108 ax.set_xlabel('年内日序'); ax.set_ylabel('日购电费/万元'); M.despine(ax); M.grid(ax)
109 ax.legend(frameon=False, ncol=3, loc='upper_right')
110 M.save_fig(fig, BASE, '图1_全年日购电费对比.png')
111
112 # 图2：全年每日首末SOC轨迹
113 fig, ax = plt.subplots(figsize=(11,4.0))
114 ax.plot(day_idx, E[:,0], color=M.C_SOC, lw=1.3, label='每日0:00储电量')
115 ax.axhline(M.SOC_MIN,color='gray',ls=':',lw=1); ax.axhline(M.SOC_MAX,color='gray',ls='--',lw=1)
116 ax.set_xlabel('年内日序'); ax.set_ylabel('储电量/kWh'); M.despine(ax); M.grid(ax)
117 ax.legend(frameon=False); ax.set_ylim(0,11500)
118 M.save_fig(fig, BASE, '图2_全年储电量变化轨迹.png')
119
120 # 图3：月度购电费/购电量柱状
121 month_fee = pd.Series(day_fee).groupby([pd.Timestamp(d).month for d in dates]).sum()
122 month_base = pd.Series(day_fee_base).groupby([pd.Timestamp(d).month for d in dates]).sum()
123 fig, ax = plt.subplots(figsize=(10,4.2))
124 xm = np.arange(12); w=0.4
125 ax.bar(xm-w/2, month_base.values/1e4, w, color='#cccccc', label='无储能', edgecolor='white')
126 ax.bar(xm+w/2, month_fee.values/1e4, w, color=M.C_CH, label='储能优化', edgecolor='white')
127 ax.set_xticks(xm); ax.set_xticklabels([f'{m}月' for m in range(1,13)])
128 ax.set_ylabel('月度购电费/万元'); M.despine(ax); M.grid(ax); ax.legend(frameon=False)
129 M.save_fig(fig, BASE, '图3_月度购电费对比.png')
130
131 # 图4：四个指定日调度曲线 2x2
132 fig, axes = plt.subplots(2,2,figsize=(12,7))
133 for ax,kd in zip(axes.flat,key_dates):
134     di=key_info[kd]['idx']
135     ax.plot(HOURS,load_y[di][NAT],color=M.C_LOAD,lw=1.5,label='负载')
136     ax.plot(HOURS,pv_y[di][NAT],color=M.C_PV,lw=1.5,label='光伏')
137     ax.plot(HOURS,g[di][NAT],color=M.C_GRID,lw=1.5,label='购电')
138     axb=ax.twinx(); axb.plot(HOURS,E[di][NAT],color=M.C_SOC,lw=1.3,ls='--',label='SOC')
139     axb.set_ylim(0,11500); M.despine(axb); axb.set_ylabel('SOC/kWh',fontsize=8)
140     ax.set_title(kd,fontsize=10); ax.set_xlim(0,24); ax.set_xticks(range(0,25,4))
141     M.despine(ax); M.grid(ax); ax.set_xlabel('时刻/h',fontsize=8)
142 axes[0,0].legend(frameon=False,ncol=3,fontsize=8,loc='upper_left')
143 plt.tight_layout()
144 M.save_fig(fig, BASE, '图4_指定日调度曲线.png')
145
146 # 图5：典型夏季日(6.21)供电构成堆叠
147 di = key_info['2025-06-21']['idx']
148 fig, ax = plt.subplots(figsize=(10,4.2))

```

```

149 ax.stackplot(HOURS, pv_y[di][NAT], g[di][NAT], d[di][NAT],
150             labels=['光伏','购电','储能放电'], colors=[M.C_PV,M.C_GRID,M.C_DIS], alpha=0.85)
151 ax.plot(HOURS, load_y[di][NAT]+c[di][NAT], color='black', lw=1.4, ls='--', label='负载+充电')
152 ax.set_xlim(0,24); ax.set_xticks(range(0,25,2)); ax.set_xlabel('时刻/h'); ax.set_ylabel('功率/
kW')
153 M.despine(ax); M.grid(ax); ax.legend(frameon=False, ncol=4, loc='upper_left')
154 M.save_fig(fig, BASE, '图5_夏季典型日供电构成.png')
155
156 # 图6: 弃光与紧急购电统计
157 fig, axes = plt.subplots(1,2,figsize=(11,3.8))
158 axes[0].bar(day_idx, day_cur, color=M.C_PV, width=1.0)
159 axes[0].set_title('全年逐日弃光电量'); axes[0].set_xlabel('年内日序'); axes[0].set_ylabel('弃
光/kWh')
160 M.despine(axes[0]); M.grid(axes[0])
161 axes[1].bar(day_idx, day_emg, color=M.C_EMG, width=1.0)
162 axes[1].set_title('全年逐日紧急购电量(确定型最优下为0)'); axes[1].set_xlabel('年内日序'); axes
[1].set_ylabel('紧急购电/kWh')
163 M.despine(axes[1]); M.grid(axes[1])
164 plt.tight_layout()
165 M.save_fig(fig, BASE, '图6_弃光与紧急购电统计.png')
166
167 # 图7: 充放电量季节关系散点
168 fig, ax = plt.subplots(figsize=(6.5,5))
169 sc=ax.scatter(day_charge, day_disch, c=range(ND), cmap='viridis', s=14, alpha=0.8)
170 lim=max(day_charge.max(), day_disch.max())
171 ax.plot([0,lim], [0,lim*0.81], color='red', ls='--', lw=1.2, label='放电=0.81×充电(综合效率)')
172 ax.set_xlabel('日充电量/kWh'); ax.set_ylabel('日放电量/kWh'); M.despine(ax); M.grid(ax)
173 ax.legend(frameon=False); plt.colorbar(sc, label='年内日序')
174 M.save_fig(fig, BASE, '图7_日充放电量效率关系.png')
175
176 print('问题二全部图表已生成并同步')

```

全年购电量 2265.89 万 kWh、购电费 1375.13 万元，较无储能 1829.86 万元节省 24.85%，紧急购电量为 0，弃光 99.02 万 kWh。

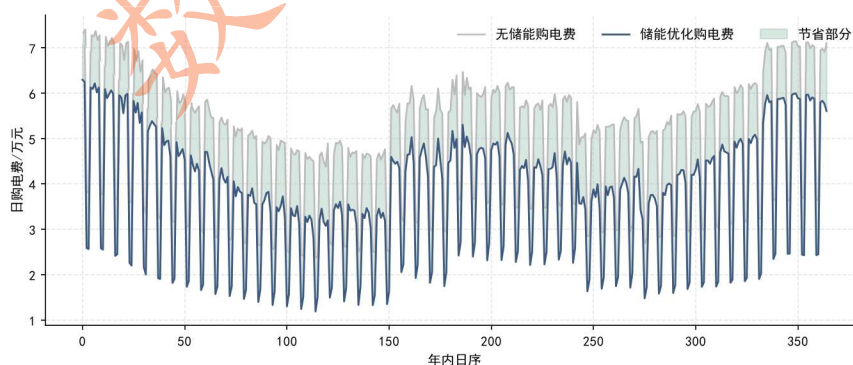


图 5: 问题二全年逐日购电费

3.3 问题三：预报驱动的计划—调整滚动调度

问题背景与具体分析

问题三考虑光伏只能依赖含误差的预报，每天 0/6/12/18 点各发布一次未来 24 小时整点预报。0 点制定计划，其余三次滚动调整；计划高于调整的差额按 0.5 倍电价违约，调整高于计划按 1.5 倍计费，实际短缺按 5 倍紧急购电。难点是分段线性偏差费用的线性化、已执行段冻结与当前 SOC 递推，以及定量评估多时刻预报的价值。

方法适用性分析

随机规划缺乏误差分布数据、场景规模膨胀；鲁棒优化偏保守且浪费已知预报；一次性刚性计划无法响应更新信息。后退时域滚动优化在每次新预报到达时只对剩余时段重解线性规划、冻结已执行段，天然契合“计划—调整”流程，配合正负偏差辅助变量即可精确线性化分段费用，兼顾前瞻性与灵活性，故采用之。

思路图

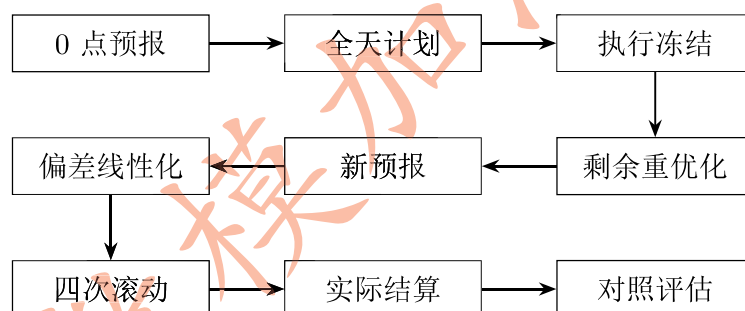


图 6: 问题三思路图

建模与模型示意

设计划购电 p_t 、调整购电 q_t ，引入 $u_t = (q_t - p_t)^+$ 、 $v_t = (p_t - q_t)^+$ ，购电侧费用为

$$C_{\text{grid}} = \sum_t p_t^e (p_t + 1.5u_t - 0.5v_t) \Delta t, \quad C_{\text{emg}} = \sum_t 5p_t^e v_t \Delta t, \quad (3)$$

在分界 36/72/108 处依次以后退时域方式重解剩余时段 LP，已执行段冻结，当前 SOC 由已执行充放递推。



核心代码与结果

```

1  # -*- coding: utf-8 -*-
2  """
3  问题三：光伏预报驱动的计划—调整滚动优化
4  =====
5  每天0:00用0点光伏预报制定计划购电策略，6/12/18:00用新预报后退时域
6  滚动调整；最终按附件2实际光伏结算：计划>调整部分罚0.5倍电价、调整>计划
7  部分按1.5倍、实际短缺按5倍紧急购电。SOC跨日连续。并对比"仅0点计划
8  不调整"方案，定量回答是否需要引入其他时刻预报。
9  """
10 import os, sys, time
11 import numpy as np
12 import pandas as pd
13 import matplotlib.pyplot as plt
14
15 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
16 import 微网公共模块 as M
17
18 BASE = os.path.dirname(os.path.abspath(__file__))
19 NAT = [143] + list(range(143)); HOURS = np.arange(144)/6.0
20
21 # ===== 第一阶段：计算 =====
22 td = M.load_typical_day(); yr = M.load_year_actual(); days_f, fcst = M.load_forecast()
23 dates = yr['dates']; ND = len(dates); price = td['price']
24 load_y, pv_y = yr['load'], yr['pv']
25
26 # 全年滚动
27 P = np.zeros((ND,144)); Q = np.zeros((ND,144)); C = np.zeros((ND,144))
28 D = np.zeros((ND,144)); EMG = np.zeros((ND,144)); CUR = np.zeros((ND,144))
29 Edaily = np.zeros((ND,145)); Stage = np.zeros((ND,144),dtype=int)
30 t0=time.time(); E_start = M.SOC_INIT
31 for di in range(ND):
32     out = M.rolling_dispatch_day(price, load_y[di], pv_y[di], fcst[di], E_start)
33     P[di]=out['p']; Q[di]=out['q']; C[di]=out['c']; D[di]=out['d']
34     EMG[di]=out['e']; CUR[di]=out['curtail']; Edaily[di]=out['E']; Stage[di]=out['stage']
35     E_start = out['E'][-1] # 跨日传递
36 print('全年滚动耗时%.1fs' % (time.time()-t0))
37
38 # 逐日费用（滚动调整方案）
39 grid_fee=np.zeros(ND); emg_fee=np.zeros(ND); plan_fee=np.zeros(ND); adj_pen=np.zeros(ND)
40 total_fee=np.zeros(ND); up_qty=np.zeros(ND); down_qty=np.zeros(ND)
41 for di in range(ND):
42     f=M.deviation_fee(price,P[di],Q[di],EMG[di])
43     grid_fee[di]=f['grid_fee']; emg_fee[di]=f['emg_fee']; plan_fee[di]=f['plan_fee']
44     adj_pen[di]=f['adj_penalty']; total_fee[di]=f['total']
45     up_qty[di]=f['up_qty']; down_qty[di]=f['down_qty']
46
47 # 对照方案：仅0点计划、全天不调整，按实际光伏结算
48 NA_EMG=np.zeros((ND,144)); na_fee=np.zeros(ND)
49 for di in range(ND):
50     # 0点计划的充放：重算一次0点优化（与滚动内部一致），用其c,d
51     pv0 = M.forecast_to_10min(fcst[di,0],0)
52     r0 = M.solve_dispatch(price, load_y[di], pv0, E_start=Edaily[di,0], E_end=None, emergency=
        True)
53     e_na, sur_na = M.settle_actual(load_y[di], pv_y[di], r0['g'], r0['c'], r0['d'])

```

```

54 NA_EMG[di]=e_na
55 na_fee[di]=np.sum(price*r0['g']*M.DT)+M.EMERGENCY_RATE*np.sum(price*e_na*M.DT)
56
57 print('='*60); print('问题三全年结果（滚动调整方案）'); print('='*60)
58 print('全年计划购电费: %.2f万元' % (plan_fee.sum()/1e4))
59 print('全年电网结算费: %.2f万元' % (grid_fee.sum()/1e4))
60 print('全年调整附加费: %.2f万元' % (adj_pen.sum()/1e4))
61 print('全年紧急购电费: %.2f万元' % (emg_fee.sum()/1e4))
62 print('全年总费用(滚动): %.2f万元' % (total_fee.sum()/1e4))
63 print('全年总费用(仅0点不调整): %.2f万元' % (na_fee.sum()/1e4))
64 print('滚动调整相对不调整节省: %.2f万元(%.2f%)' % ((na_fee.sum()-total_fee.sum())/1e4,
65 (na_fee.sum()-total_fee.sum())/na_fee.sum()*100))
66 print('全年紧急购电量(滚动) %.1f kWh, (不调整) %.1f kWh' % (EMG.sum()/6, NA_EMG.sum()/6))
67 print('全年上调量 %.1f kWh, 下调量 %.1f kWh' % (up_qty.sum(), down_qty.sum()))
68 print('SOC范围 %.1f~%.1f' % (Edaily.min(), Edaily.max()))
69
70 # 输出区间 2.1-12.31
71 out_mask=np.array([(d.month>=2) for d in dates]); idx=np.where(out_mask)[0]
72 dates_out=[dates[i] for i in idx]
73 price_o=np.tile(price,(len(idx),1))
74 out_path=M.write_result_year(BASE,'result3.xlsx',dates_out,P[idx],C[idx],D[idx],Edaily[idx],
75 e_year=EMG[idx],adj_year=Q[idx],price_for_fee=price_o)
76 # 计划购电量sheet=0点计划P, 调整购电量sheet=最终调整Q, 充放电=最终C/D, 紧急=EMG
77 print('已写出', out_path)
78
79 # 指定日结果
80 key_dates=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
81 tab1_slots=['10:00','12:00','14:00','16:00','18:00','20:00']
82 key_info={}
83 for kd in key_dates:
84     di=dates.index(pd.Timestamp(kd).date()); key_info[kd]=di
85     print(f'\n{kd}: 计划购电{P[di].sum()/6:.1f} 调整购电{Q[di].sum()/6:.1f} 紧急{EMG[di].sum()/6:.1f} kWh 总费{total_fee[di]:.1f}元 不调整费{na_fee[di]:.1f}元')
86     print('最终购电表1:', {s:round(Q[di,M.slot_index_by_hm(s)]/6,2) for s in tab1_slots})
87
88 # 保存CSV
89 stat=pd.DataFrame({'日期':dates,'计划购电费':plan_fee,'电网结算费':grid_fee,'调整附加费':
90 adj_pen,
91 '紧急购电费':emg_fee,'滚动总费用':total_fee,'不调整总费用':na_fee,
92 '上调量':up_qty,'下调量':down_qty,
93 '滚动紧急电量':EMG.sum(1)/6,'不调整紧急电量':NA_EMG.sum(1)/6})
94 M.save_csv(stat,BASE,'问题三_全年滚动调度统计.csv')
95
96 # ===== 第二阶段: 绘图 =====
97 di_x=np.arange(ND)
98 # 图1: 滚动 vs 不调整 全年总费用
99 fig,ax=plt.subplots(figsize=(11,4.2))
100 ax.plot(di_x,na_fee/1e4,color='#d62828',lw=1.2,label='仅0点计划(不调整)')
101 ax.plot(di_x,total_fee/1e4,color=M.C_LOAD,lw=1.3,label='四次滚动调整')
102 ax.fill_between(di_x,total_fee/1e4,na_fee/1e4,color=M.C_PV,alpha=0.3,label='调整节省')
103 ax.set_xlabel('年内日序'); ax.set_ylabel('日总费用/万元'); M.despine(ax); M.grid(ax)
104 ax.legend(frameon=False,ncol=3,loc='upper_right')
105 M.save_fig(fig,BASE,'图1_滚动调整与不调整全年费用对比.png')
106
107 # 图2: 费用构成堆叠(日均/月度)
108 month_grp=np.array([d.month for d in dates])

```

```
108 mg=lambda v: pd.Series(v).groupby(month_grp).sum().reindex(range(1,13)).values
109 m_grid,m_adj,m_emg=mg(grid_fee),mg(adj_pen),mg(emg_fee)
110 fig,ax=plt.subplots(figsize=(10,4.2)); xm=np.arange(12)
111 ax.bar(xm,m_grid/1e4,color=M.C_LOAD,label='电网购电费',edgecolor='white')
112 ax.bar(xm,m_adj/1e4,bottom=m_grid/1e4,color=M.C_GRID,label='调整附加费',edgecolor='white')
113 ax.bar(xm,m_emg/1e4,bottom=(m_grid+m_adj)/1e4,color=M.C_EMG,label='紧急购电费',edgecolor='
    white')
114 ax.set_xticks(xm);ax.set_xticklabels([f'{m}月' for m in range(1,13)])
115 ax.set_ylabel('月度费用/万元');M.despine(ax);M.grid(ax);ax.legend(frameon=False,ncol=3)
116 M.save_fig(fig,BASE,'图2_月度三类费用构成.png')
117
118 # 图3: 典型日(3.20)计划vs调整购电曲线 + 实际光伏vs预报
119 di=key_info['2025-03-20']
120 pv0=M.forecast_to_10min(fcst[di,0],0)
121 fig,ax=plt.subplots(figsize=(10,4.4))
122 ax.plot(HOURS,pv0[NAT],color=M.C_PV,lw=1.6,ls='--',label='0点光伏预报')
123 ax.plot(HOURS,pv_y[di][NAT],color=M.C_PV,lw=2.0,label='光伏实际')
124 ax.plot(HOURS,P[di][NAT],color='#999999',lw=1.5,label='0点计划购电')
125 ax.plot(HOURS,Q[di][NAT],color=M.C_LOAD,lw=1.8,label='滚动调整购电')
126 ax.fill_between(HOURS,0,EMG[di][NAT],color=M.C_EMG,alpha=0.3,label='紧急购电')
127 ax.set_xlim(0,24);ax.set_xticks(range(0,25,2));ax.set_xlabel('时刻/h');ax.set_ylabel('功率/kW'
    )
128 M.despine(ax);M.grid(ax);ax.legend(frameon=False,ncol=3,fontsize=8,loc='upper_left')
129 M.save_fig(fig,BASE,'图3_典型日计划与调整购电对比.png')
130
131 # 图4: 四次滚动更新过程(以3.20为例,各预报段用不同底色)
132 fig,ax=plt.subplots(figsize=(10,4.2))
133 seg_colors=['#e9edf5','#dce7f0','#e9edf5','#dce7f0']
134 for k in range(4):
135     s=[0,36,72,108][k]; e=[36,72,108,144][k]
136     ax.axvspan(s/6,e_/6,color=seg_colors[k],zorder=0)
137 ax.plot(HOURS,Q[di][NAT],color=M.C_LOAD,lw=1.8,label='最终调整购电')
138 ax.plot(HOURS,P[di][NAT],color=M.C_EMG,lw=1.3,ls='--',label='0点计划购电')
139 for k,txt in enumerate(['0点预报','6点调整','12点调整','18点调整']):
140     ax.text([0,6,12,18][k]+2.6,ax.get_ylim()[1]*0.95,txt,ha='center',fontsize=9,color='#555')
141 ax.set_xlim(0,24);ax.set_xticks(range(0,25,2));ax.set_xlabel('时刻/h');ax.set_ylabel('购电功率
    /kW')
142 M.despine(ax);M.grid(ax);ax.legend(frameon=False,loc='upper_left')
143 M.save_fig(fig,BASE,'图4_四次滚动更新时段划分.png')
144
145 # 图5: 预报误差分布(0点预报 vs 实际,按小时)
146 err_all=[]
147 for di2 in range(ND):
148     pv0=M.forecast_to_10min(fcst[di2,0],0)
149     err_all.append((pv0-pv_y[di2]))
150 err_all=np.array(err_all).flatten()/1
151 fig,ax=plt.subplots(figsize=(8,4.2))
152 ax.hist(err_all,bins=80,color=M.C_CH,edgecolor='white',alpha=0.85)
153 ax.axvline(0,color='black',lw=1)
154 ax.set_xlabel('光伏预报误差(预报-实际)/kW');ax.set_ylabel('时段频数');M.despine(ax);M.grid(ax)
155 M.save_fig(fig,BASE,'图5_光伏预报误差分布.png')
156
157 # 图6: 紧急购电量 滚动vs不调整
158 fig,ax=plt.subplots(figsize=(11,4.0))
159 ax.plot(di_x,NA_EMG.sum(1)/6,color=M.C_EMG,lw=1.2,label='不调整紧急购电量')
160 ax.plot(di_x,EMG.sum(1)/6,color=M.C_LOAD,lw=1.3,label='滚动调整紧急购电量')
```

```
161 ax.set_xlabel('年内日序');ax.set_ylabel('日紧急购电量/kWh');M.despine(ax);M.grid(ax);ax.legend(
    (frameon=False)
162 M.save_fig(fig,BASE,'图6_紧急购电量对比.png')
163
164 # 图7：调整方向（上调/下调）逐日
165 fig,ax=plt.subplots(figsize=(11,3.8))
166 ax.bar(di_x,up_qty,color=M.C_GRID,width=1.0,label='上调量(调整>计划)')
167 ax.bar(di_x,-down_qty,color=M.C_CH,width=1.0,label='下调量(计划>调整)')
168 ax.axhline(0,color='black',lw=0.8)
169 ax.set_xlabel('年内日序');ax.set_ylabel('调整电量/kWh');M.despine(ax);M.grid(ax);ax.legend(
    (frameon=False)
170 M.save_fig(fig,BASE,'图7_逐日调整方向与规模.png')
171
172 print('问题三全部图表已生成并同步')
```

滚动调整全年总费 1890.30 万元，其中电网结算 1439.01 万、调整附加 35.15 万、紧急购电 451.29 万；不调整为 1934.10 万元，滚动节省 43.80 万元（2.26%），紧急购电量由 125.30 万降至 105.16 万 kWh，证明多时刻预报确有价值。

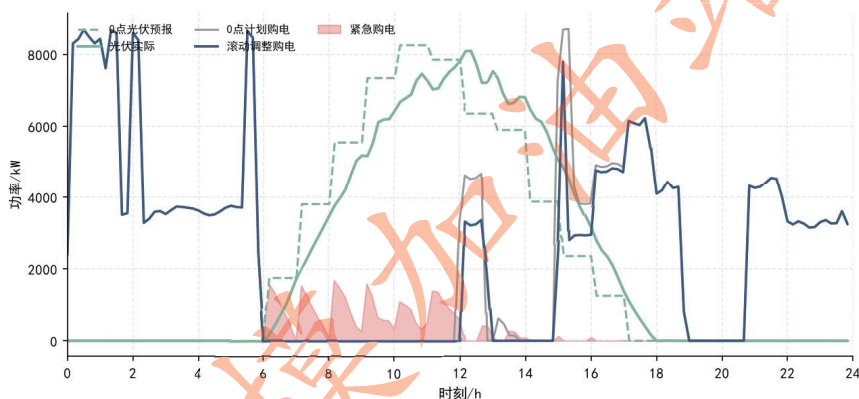


图 7: 问题三典型日计划与调整对比

3.4 问题四：实时波动电价下的调度

问题背景与具体分析

问题四将固定电价替换为附件 4 的逐日波动电价，分别重算问题二（确定型，result4-2）与问题三（滚动型，result4-3），并做四场景对比。模型结构不变、仅价格系数改变，难点在于保证四场景口径一致、结论可比。

方法适用性分析

价格阈值启发式在逐日不同的价格下难以自适应，电价随机规划与“日前已知当日电价”的信息假设不符；线性规划以电价为目标系数，价格波动只改变目标向量而约束

结构不变，求解器自动实现低价多充、高价多放，框架可无缝复用，故沿用线性规划与滚动优化。

思路图

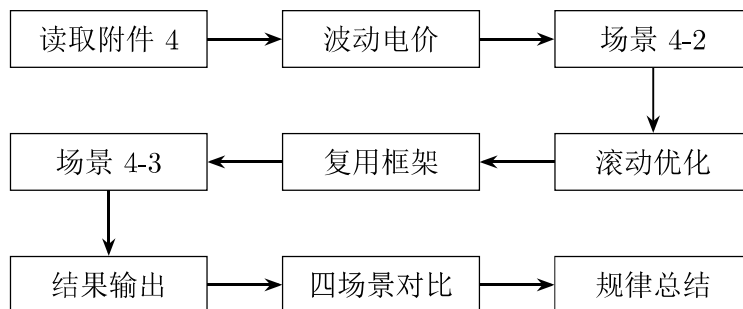
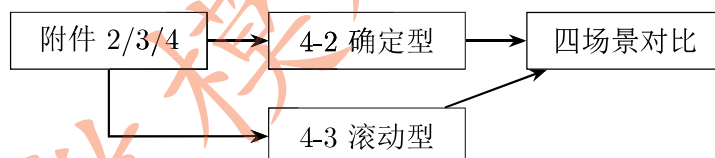


图 8: 问题四思路图

建模与模型示意

将偏差结算与目标中的电价 p_t^e 全部替换为附件 4 波动值：

$$Z_{4-2} = \sum_t (p_t g_t + 5p_t e_t) \Delta t, \quad Z_{4-3} = \sum_t p_t [(p_t^{\text{plan}} - v_t) + 1.5u_t + 0.5v_t + 5e_t] \Delta t. \quad (4)$$



核心代码与结果

```

1 # -*- coding: utf-8 -*-
2 """
3 问题四：实时波动电价下重算问题二、问题三
4 =====
5 将附件1固定电价替换为附件4逐日波动电价，其余数据(附件2负载/光伏、
6 附件3预报)不变：result4-2_对应波动价下的问题二(全年确定型LP)，
7 result4-3_对应波动价下的问题三(四次滚动调整)，并与固定电价四场景对比。
8 """
9 import os, sys, time
10 import numpy as np
11 import pandas as pd
12 import matplotlib.pyplot as plt
13
14 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
15 import 微网公共模块 as M
16

```

```

17 BASE = os.path.dirname(os.path.abspath(__file__))
18 NAT=[143]+list(range(143)); HOURS=np.arange(144)/6.0
19
20 td=M.load_typical_day(); yr=M.load_year_actual(); _,fcst=M.load_forecast()
21 dates=yr['dates']; ND=len(dates)
22 load_y,pv_y,price_y=yr['load'],yr['pv'],yr['price']
23 fixed_price=td['price']
24
25 # ===== 场景 4-2: 波动电价全年确定型 LP =====
26 print('====问题4-2_波动电价全年确定型====')
27 t0=time.time()
28 res42=M.solve_dispatch(price_y.reshape(-1),load_y.reshape(-1),pv_y.reshape(-1),
29                        E_start=M.SOC_INIT,E_end=None,emergency=True)
30 print('求解耗时%.2fs'%(time.time()-t0))
31 g42=res42['g'].reshape(ND,144); c42=res42['c'].reshape(ND,144)
32 d42=res42['d'].reshape(ND,144); e42=res42['e'].reshape(ND,144); cur42=res42['cur'].reshape(ND
    ,144)
33 E42=np.array([res42['E'][di*144:di*144+145] for di in range(ND)])
34 fee42=np.array([np.sum(price_y[di]*g42[di]*M.DT for di in range(ND))])
35 qty42=g42.sum(1)*M.DT
36 base42=np.array([np.sum(price_y[di]*np.maximum(load_y[di]-pv_y[di],0)*M.DT for di in range(ND
    ))])
37 print('波动价确定型_全年购电费%.2f万元,紧急%.2f_kWh,弃光%.1f_kWh'%(fee42.sum()/1e4,e42.
    sum()/6,cur42.sum()/6))
38 print('无储能对照%.2f万元,节省%.2f%%'%(base42.sum()/1e4,(base42.sum()-fee42.sum())/base42.
    sum()*100))
39
40 # ===== 场景 4-3: 波动电价四次滚动 =====
41 print('====问题4-3_波动电价全年滚动====')
42 P43=np.zeros((ND,144)); Q43=np.zeros((ND,144)); C43=np.zeros((ND,144)); D43=np.zeros((ND,144))
43 EMG43=np.zeros((ND,144)); E43=np.zeros((ND,145))
44 t0=time.time(); Es=M.SOC_INIT
45 for di in range(ND):
46     o=M.rolling_dispatch_day(price_y[di],load_y[di],pv_y[di],fcst[di],Es)
47     P43[di]=o['p']; Q43[di]=o['q']; C43[di]=o['c']; D43[di]=o['d']; EMG43[di]=o['e']; E43[di]=o['E']
48     Es=o['E'][-1]
49 print('滚动耗时%.2fs'%(time.time()-t0))
50 tot43=np.zeros(ND); grid43=np.zeros(ND); adj43=np.zeros(ND); emg43=np.zeros(ND)
51 na43=np.zeros(ND); NAEMG43=np.zeros((ND,144))
52 for di in range(ND):
53     f=M.deviation_fee(price_y[di],P43[di],Q43[di],EMG43[di])
54     tot43[di]=f['total']; grid43[di]=f['grid_fee']; adj43[di]=f['adj_penalty']; emg43[di]=f['
        emg_fee']
55     pv0=M.forecast_to_10min(fcst[di],0)
56     r0=M.solve_dispatch(price_y[di],load_y[di],pv0,E_start=E43[di,0],E_end=None,emergency=True
        )
57     e_na,_=M.settle_actual(load_y[di],pv_y[di],r0['g'],r0['c'],r0['d']); NAEMG43[di]=e_na
58     na43[di]=np.sum(price_y[di]*r0['g']*M.DT)+M.EMERGENCY_RATE*np.sum(price_y[di]*e_na*M.DT)
59 print('波动价滚动_全年总费%.2f万元_电网%.2f调整%.2f紧急%.2f'%(tot43.sum()/1e4,grid43.sum
    ()/1e4,adj43.sum()/1e4,emg43.sum()/1e4))
60 print('波动价不调整%.2f万元,滚动节省%.2f万元(%.2f%%'%(na43.sum()/1e4,(na43.sum()-tot43.
    sum())/1e4,(na43.sum()-tot43.sum())/na43.sum()*100))
61
62 # ===== 读取固定价结果(问题二/三CSV)用于四场景对比 =====
63 p2=pd.read_csv(os.path.join(M.SOLVE_DIR,'问题二','问题二_全年逐日调度统计.csv'))

```

```

64 p3=pd.read_csv(os.path.join(M.SOLVE_DIR,'问题三','问题三_全年滚动调度统计.csv'))
65 scen={
66     '问题二\n固定价确定型': p2['日购电费_元'].sum(),
67     '问题4-2\n波动价确定型': fee42.sum(),
68     '问题三\n固定价滚动': p3['滚动总费用'].sum(),
69     '问题4-3\n波动价滚动': tot43.sum(),
70 }
71 print('\n====四场景全年总费用(万元)====')
72 for k,v in scen.items(): print(f'{k.replace(chr(10),"")}:_{v/1e4:.2f}')
73
74 # ===== 写出 result4-2 / result4-3 =====
75 out_mask=np.array([(d.month>=2) for d in dates]); idx=np.where(out_mask)[0]
76 dates_out=[dates[i] for i in idx]
77 op1=M.write_result_year(BASE,'result4-2.xlsx',dates_out,g42[idx],c42[idx],d42[idx],E42[idx],
78                        e_year=e42[idx],price_for_fee=price_y[idx])
79 op2=M.write_result_year(BASE,'result4-3.xlsx',dates_out,P43[idx],C43[idx],D43[idx],E43[idx],
80                        e_year=EMG43[idx],adj_year=Q43[idx],price_for_fee=price_y[idx])
81 print('已写出',op1);print('已写出',op2)
82
83 # CSV
84 pd.DataFrame({'日期':dates,'波动价购电费':fee42,'波动价购电量':qty42,'无储能费':base42}).
85     to_csv(
86         os.path.join(BASE,'问题四_波动价确定型统计.csv'),index=False,encoding='utf-8-sig')
87 pd.DataFrame({'日期':dates,'电网费':grid43,'调整附加':adj43,'紧急费':emg43,'滚动总费':tot43,'
88     不调整费':na43}).to_csv(
89     os.path.join(BASE,'问题四_波动价滚动统计.csv'),index=False,encoding='utf-8-sig')
90
91 # ===== 绘图 =====
92 di_x=np.arange(ND)
93 # 图1: 固定电价 vs 波动电价曲线 (选3天展示波动)
94 fig,ax=plt.subplots(figsize=(10,4.2))
95 for di,lab,al in [(0,'1月1日',0.9),(171,'6月21日',0.9),(354,'12月21日',0.9)]:
96     ax.plot(HOURS,price_y[di][NAT],lw=1.5,alpha=al,label=f'波动电价_{lab}')
97 ax.plot(HOURS,fixed_price[NAT],color='black',lw=1.8,ls='--',label='附件1固定电价')
98 ax.set_xlim(0,24);ax.set_xticks(range(0,25,2));ax.set_xlabel('时刻/h');ax.set_ylabel('电价/(元/kWh)')
99 M.despine(ax);M.grid(ax);ax.legend(frameon=False,ncol=2)
100 M.save_fig(fig,BASE,'图1_固定与波动电价对比.png')
101
102 # 图2: 波动价确定型 全年日购电费 vs 无储能
103 fig,ax=plt.subplots(figsize=(11,4.0))
104 ax.plot(di_x,base42/1e4,color='#cccccc',lw=1.2,label='无储能')
105 ax.plot(di_x,fee42/1e4,color=M.C_LOAD,lw=1.3,label='波动价储能优化')
106 ax.fill_between(di_x,fee42/1e4,base42/1e4,color=M.C_PV,alpha=0.3)
107 ax.set_xlabel('年内日序');ax.set_ylabel('日购电费/万元');M.despine(ax);M.grid(ax);ax.legend(
108     frameon=False)
109 M.save_fig(fig,BASE,'图2_波动价确定型全年费用.png')
110
111 # 图3: 四场景全年总费用柱状对比
112 fig,ax=plt.subplots(figsize=(8.5,4.6))
113 ks=list(scen.keys());vs=[scen[k]/1e4 for k in ks]
114 bars=ax.bar(range(4),vs,color=[M.C_LOAD,M.C_CH,M.C_GRID,M.C_DIS],width=0.6,edgecolor='white')
115 for b,v in zip(bars,vs): ax.text(b.get_x()+b.get_width()/2,v+15,f'{v:.1f}',ha='center',
116     fontsize=10)
117 ax.set_xticks(range(4));ax.set_xticklabels(ks,fontsize=9);ax.set_ylabel('全年总费用/万元')
118 M.despine(ax);M.grid(ax);ax.set_ylim(0,max(vs)*1.12)

```

```
115 M.save_fig(fig,BASE,'图3_四场景全年费用对比.png')
116
117 # 图4: 固定vs波动 确定型日费用差
118 fig,ax=plt.subplots(figsize=(11,3.8))
119 diff=fee42-p2['日购电费_元'].values
120 ax.bar(di_x,diff/1e4,color=np.where(diff>=0,M.C_EMG,M.C_CH),width=1.0)
121 ax.axhline(0,color='black',lw=0.8)
122 ax.set_xlabel('年内日序');ax.set_ylabel('(波动价-固定价)日费用/万元');M.despine(ax);M.grid(ax)
123 M.save_fig(fig,BASE,'图4_波动与固定电价日费用差.png')
124
125 # 图5: 波动价滚动 vs 不调整
126 fig,ax=plt.subplots(figsize=(11,4.0))
127 ax.plot(di_x,na43/1e4,color=M.C_EMG,lw=1.2,label='仅0点计划不调整')
128 ax.plot(di_x,tot43/1e4,color=M.C_LOAD,lw=1.3,label='四次滚动调整')
129 ax.fill_between(di_x,tot43/1e4,na43/1e4,color=M.C_PV,alpha=0.3,label='滚动节省')
130 ax.set_xlabel('年内日序');ax.set_ylabel('日总费用/万元');M.despine(ax);M.grid(ax);ax.legend(
    frameon=False)
131 M.save_fig(fig,BASE,'图5_波动价滚动与不调整对比.png')
132
133 # 图6: 典型日(12.21)波动价下调度
134 di=dates.index(pd.Timestamp('2025-12-21').date())
135 fig,ax1=plt.subplots(figsize=(10,4.4))
136 ax1.plot(HOURS,load_y[di][NAT],color=M.C_LOAD,lw=1.5,label='负载')
137 ax1.plot(HOURS,pv_y[di][NAT],color=M.C_PV,lw=1.5,label='光伏')
138 ax1.plot(HOURS,g42[di][NAT],color=M.C_GRID,lw=1.6,label='购电(确定型)')
139 ax1.set_xlim(0,24);ax1.set_xticks(range(0,25,2));ax1.set_xlabel('时刻/h');ax1.set_ylabel('功率/kW')
140 ax2=ax1.twinx();ax2.plot(HOURS,price_y[di][NAT],color=M.C_PRICE,lw=1.4,ls='--',label='波动电价')
141 ax2.set_ylabel('电价/(元/kWh)',color=M.C_PRICE);M.despine(ax2);M.despine(ax1);M.grid(ax1)
142 lines=ax1.get_lines()+ax2.get_lines();ax1.legend(lines,[l.get_label() for l in lines],frameon=False,ncol=2,fontsize=8)
143 M.save_fig(fig,BASE,'图6_波动价冬季典型日调度.png')
144
145 # 图7: 月度费用四场景
146 month=np.array([d.month for d in dates])
147 mg=lambda v: pd.Series(v).groupby(month).sum().reindex(range(1,13)).values/1e4
148 m2=mg(p2['日购电费_元'].values);m42=mg(fee42);m3=mg(p3['滚动总费用'].values);m43=mg(tot43)
149 fig,ax=plt.subplots(figsize=(10,4.2));xm=np.arange(12);w=0.2
150 ax.bar(xm-1.5*w,m2,w,label='固定确定型',color=M.C_LOAD,edgecolor='white')
151 ax.bar(xm-0.5*w,m42,w,label='波动确定型',color=M.C_CH,edgecolor='white')
152 ax.bar(xm+0.5*w,m3,w,label='固定滚动',color=M.C_GRID,edgecolor='white')
153 ax.bar(xm+1.5*w,m43,w,label='波动滚动',color=M.C_DIS,edgecolor='white')
154 ax.set_xticks(xm);ax.set_xticklabels([f'{m}月' for m in range(1,13)]);ax.set_ylabel('月度费用/万元')
155 M.despine(ax);M.grid(ax);ax.legend(frameon=False,ncol=4,fontsize=8)
156 M.save_fig(fig,BASE,'图7_月度四场景费用对比.png')
157
158 print('问题四全部图表已生成并同步')
```

波动价确定型 1449.48 万元（省 24.33%），波动价滚动 1992.75 万元（较不调整省 43.72 万元、2.15%）。四场景费用依次为 1375.13、1449.48、1890.30、1992.75 万元，预报不确定性是比电价波动更主要的成本来源。

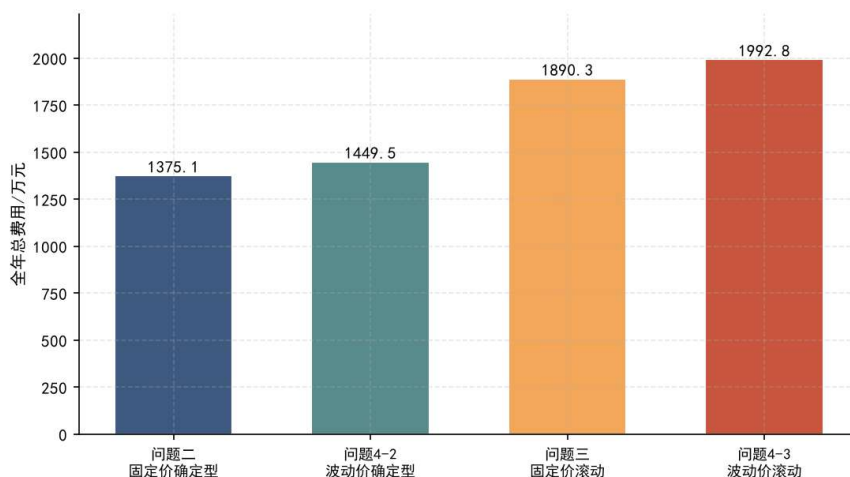


图 9: 四场景全年费用对比

4 建模方法匹配表

表 1: 各问题建模方法匹配表

问题	问题类型	主方法	备选方法	关键工具
问题一	单日确定型优化	线性规划 (LP)	动态规划/贪心	scipy HiGHS
问题二	全年大规模优化	稀疏线性规划	逐日优化/DP	稀疏矩阵 +HiGHS
问题三	不确定滚动优化	后退时域 LP	随机/鲁棒优化	滚动 + 偏差 线性化
问题四	波动价对比优化	LP/滚动复用	价格阈值法	HiGHS+ 对 照实验

5 各题输出标准

问题一

输出 result1.xlsx (144 时段计划购电量、6 段充放电量、储电量)，典型日购电量、购电费、节省率，以及电价负载光伏曲线、购电与净负荷、SOC 轨迹、充放电功率、套利时段、供电构成、有无储能对比共 7 张图。

问题二

输出 result2.xlsx (2.1—12.31 共 334 天计划购电量、充放电量、紧急购电量、全天购电量与费用), 全年购电量、购电费、节省率、弃光与紧急购电统计, 四个指定日结果, 以及全年费用、SOC 轨迹、月度对比、指定日曲线等 7 张图。

问题三

输出 result3.xlsx (计划购电量、调整购电量、最终充放电量、紧急购电量), 计划费、电网结算费、调整附加费、紧急购电费、滚动与不调整总费用及节省额, 以及滚动对比、月度费用构成、典型日计划调整、滚动时段划分、预报误差分布等 7 张图。

问题四

输出 result4-2.xlsx、result4-3.xlsx, 波动价下确定型与滚动型全年费用、节省率, 以及固定/波动电价对比、四场景费用对比、月度四场景对比等 7 张图。

6 参考文献推荐

参考文献

- [1] 钱颂迪. 运筹学 [M]. 第 4 版. 北京: 清华大学出版社, 2012.
线性规划建模与单纯形/内点法理论基础, 支撑问题一、二、四。
- [2] 丁明, 刘小平, 等. 含储能电池的微网经济运行优化 [J]. 电力系统自动化, 2014, 38(12): 1-7.
微网储能跨时段经济调度与弃光处理, 对应问题一、二。
- [3] 王成福, 等. 模型预测控制在微电网能量管理中的应用综述 [J]. 电力系统自动化, 2018, 42(10): 1-10.
后退时域滚动优化方法学, 对应问题三、四。
- [4] Bertsimas D, Tsitsiklis J N. Introduction to Linear Optimization[M]. Athena Scientific, 1997.
分段线性凸函数的辅助变量线性化与目标系数敏感性, 对应问题三、四。
- [5] 张粒子, 等. 电力市场环境下微电网经济调度研究综述 [J]. 电网技术, 2017, 41(7): 2145-2154.
实时波动电价下的微网调度背景, 对应问题四。

- [6] 王成山, 武震, 李鹏. 微电网与主动配电网相关问题分析 [J]. 中国电机工程学报, 2014, 34(19): 3004-3012.

微网能量管理与供需平衡的领域背景。

- [7] 姜启源, 谢金星, 叶俊. 数学模型 [M]. 第 5 版. 北京: 高等教育出版社, 2018.

优化建模与对照实验设计的基础参考。

- [8] Huang W, et al. Large-scale linear programming with HiGHS[CP]. Mathematical Programming Computation, 2024.

HiGHS 高性能求解器技术文献，对应全年大规模 LP 求解。

数模加油站

2026国赛国奖 1v1赛中辅导

名师授课 试听机制
合同保障 性价比高

▶▶▶ 服务详情 ◀◀◀

① 师资力量

- 指导老师均为历年国/研赛国一或美赛O/F奖获得者/其他数模竞赛国奖获得者

② 赛中服务

- 赛中为学员提供选题/思路/建模/编程/写作指导和不限次数答疑

③ 课程保障

- 试听机制、签订合同，保障学员权益
- 若未获得指定奖项，免费转为学术论文发表

扫码添加学术顾问了解课程详情



数模国赛 转学术论文

• 抢购开始时间：8月31日 •

① 福利一：优惠福利

- 活动期间前66名立减700-2800元

② 福利二：赠品福利

- 免费使用自研数模智能体
- 赠送国赛赛中vip资料
- 国一老师免费检查全文&预测奖项
- 赠送价值5000元全年通精品大班课(国赛、数竞、美赛、四六级、大英赛、蓝桥杯等)

③ 福利三：转多篇福利

- 预定两篇及以上数模转尊享折上折

详情请扫码添加学术顾问咨询

限时预定 8.31-9.9享全部福利

